

TOP 15 **MOST ASKED** **OOP QUESTIONS** in Java Interviews (2026)



@java_in.depth

If you're preparing for Java backend roles, these are must-know OOP questions.

Save this if you're preparing for Java Interviews.

Q1: WHAT IS OOP?

OOP = Object-Oriented Programming
It organizes code using objects and classes.

4 Pillars:

- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

Example:

Car object → properties + behavior

Common Mistake

Only memorizing definitions without real examples.

Q2: WHAT IS ENCAPSULATION?

Binding data + methods together.

```
class User {  
    private String name;  
  
    public String getName() { return name; }  
    public void setName(String name) { this.name = name; }  
}
```

Why?

- Data hiding

- Controlled access

Mistake

Making variables public → breaks encapsulation.

Q3: WHAT IS INHERITANCE?

Child class inherits properties from parent.

```
class Animal {  
    void eat() {}  
}  
  
class Dog extends Animal {  
    void bark() {}  
}
```

Benefits:

- Code reuse

- Hierarchy

Mistake

Using inheritance where composition is better.

Q4: WHAT IS POLYMORPHISM?

Same method, different behavior.

Types:

- Compile-time (Overloading)
- Runtime (Overriding)

```
void add(int a, int b) {}  
void add(int a, int b, int c) {}
```

Mistake

Confusing overloading vs overriding.

Q5: WHAT IS ABSTRACTION?

Hiding implementation details.

```
abstract class Vehicle {  
    abstract void start();  
}
```

Focus:

What to do (not how)

Mistake

Thinking abstraction = only abstract class
(interfaces too).

Q6: OVERLOADING VS OVERRIDING, DIFFERENCE?

Overloading:	Overriding:
Same class	Parent-child
Compile time	Runtime
Method signature changes	Same signature

Mistake

Wrong method signature in overriding.

Q7. INTERFACE VS ABSTRACT CLASS, DIFFERENCE?

Interface:

- 100% abstraction (mostly)
- Multiple inheritance

Abstract Class:

- Partial abstraction
- Can have constructors

Mistake

Using abstract class where interface fits better.

Q8: CONSTRUCTOR FLOW?

Parent constructor runs first.

```
class A {  
    A() { System.out.println("A"); }  
}  
class B extends A {  
    B() { System.out.println("B"); }  
}
```

Output: A → B

Mistake

Forgetting `super()` behavior.

Q9: THIS VS SUPER

- this → current object
- super → parent class

Use cases:

call parent constructor
access parent methods

Mistake

Using both incorrectly in constructors.

Q10: METHOD OVERRIDING RULES?

- ✓ Same method name
- ✓ Same parameters
- ✓ Cannot reduce visibility
- ✓ Can change return type (covariant)

Mistake

Trying to override static methods.

Q11: WHAT IS RUNTIME POLYMORPHISM?

Method decided at runtime.



```
Animal a = new Dog();  
a.eat();
```

Mistake

Thinking reference type decides method call.

Q12: COMPOSITION VS INHERITANCE, DIFFERENCE?

Composition (HAS-A)
Better flexibility.

```
class Engine {}  
class Car {  
    Engine engine;  
}
```

Mistake

Overusing inheritance → tight coupling.

Q13: ACCESS MODIFIERS, TYPES?

- private
- default
- protected
- public

Mistake

Not understanding package-level access.

Q14: WHAT IS OBJECT CLASS?

Parent of all classes in Java.

Common methods:

- toString()
- equals()
- hashCode()

Mistake

Not overriding equals & hashCode properly.

Q15: FINAL KEYWORD?

- final variable → constant
- final method → cannot override
- final class → cannot extend

Mistake

Confusing final with finally / finalize.

Save this if you're preparing for Java Interviews.